Reproducible Data Analysis: Airports, Monte Carlo Integration, and GenAI Prompting

Akshith Kachakayala

April 17, 2026

Table of contents

1 Busiest Airports Analysis **1**
1.1 Overview . . . 1
1.2 Data Collection and Preparation . . . 1
1.3 Summary Table . . . 2
1.4 Visualization . . . 2

2 Monte Carlo Numerical Integration **3**
2.1 Background . . . 3
2.2 Assigned Function . . . 3
2.3 Simulation and Point Generation . . . 4
2.4 Small-Multiple Visualization . . . 4

Code Appendix **5**

1 Busiest Airports Analysis

1.1 Overview

This section analyzes passenger traffic at six of the world's busiest airports from 2020 through 2025. Data are scraped directly from Wikipedia's *List of busiest airports by passenger traffic*, ensuring the analysis is reproducible and current. The six airports examined are Atlanta Hartsfield–Jackson (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Phoenix Sky Harbor (PHX), Chicago O'Hare (ORD), and London Heathrow (LHR).

1.2 Data Collection and Preparation

Two data visualizations of a table and a plot were created that compare six of the busiest passenger airports in the world over a six year time period (2020 to 2025). The six airports are

Hartsfield-Jackson Atlanta International Airport (ATL), Frankfurt Airport (FRA), Beijing Daxing International Airport (PKX), Dubai International Airport (DXB), London Heathrow Airport (LHR), and Tokyo Haneda Airport (HND)

1.3 Summary Table

The plot displays a line graph with year on the horizontal axis running from 2020 to 2025 and total passengers in millions on the vertical axis starting at zero. All the airports saw a drop in passengers due to the COVID 19 pandemic in the years 2020 and 2021. Atlanta and Dubai had the highest passenger totals in the later years, while Beijing Daxing had a slower rebound after the pandemic.

Table 1: Total annual passengers (2020–2025) for six of the world's busiest airports.

Airport Name	IATA	2021	2022	2023	2024
Hartsfield–Jackson Atlanta International Airport	ATL	42,918,685	75,704,760	93,699,630	104,653,451
O'Hare International Airport	ORD	30,860,251	54,020,399	68,340,619	73,894,226
Heathrow Airport	LHR	22,111,469		61,614,508	79,183,364
Phoenix Sky Harbor International Airport	PHX	21,928,708	38,846,713	44,397,854	48,654,432
Frankfurt Airport	FRA	18,768,601	24,812,849	48,918,482	59,355,389
Beijing Daxing International Airport	PKX		25,051,012		

Note: Airports absent from a year's top-50 ranking have missing passenger data. Highlighted rows are the thre

1.4 Visualization

Looking at both the table and the line plot together, it is clear that all six airports took a huge hit in passenger numbers in 2020 because of the COVID-19 pandemic, but they did not all recover at the same speed. Phoenix and Atlanta bounced back the fastest, which makes sense because those airports handle a lot of domestic flights within the United States, and Americans started traveling again pretty quickly once restrictions lifted. Frankfurt and Tokyo Haneda recovered more slowly because they rely more heavily on international travelers, who took longer to return. China kept its borders closed much longer than the other countries so predictably Beijing has one of the slowest progressions in passenger numbers. By 2024 and 2025 most of the airports had climbed back close to or above their pre-pandemic passenger levels, but depending on the political context and locality, some countries were slower.

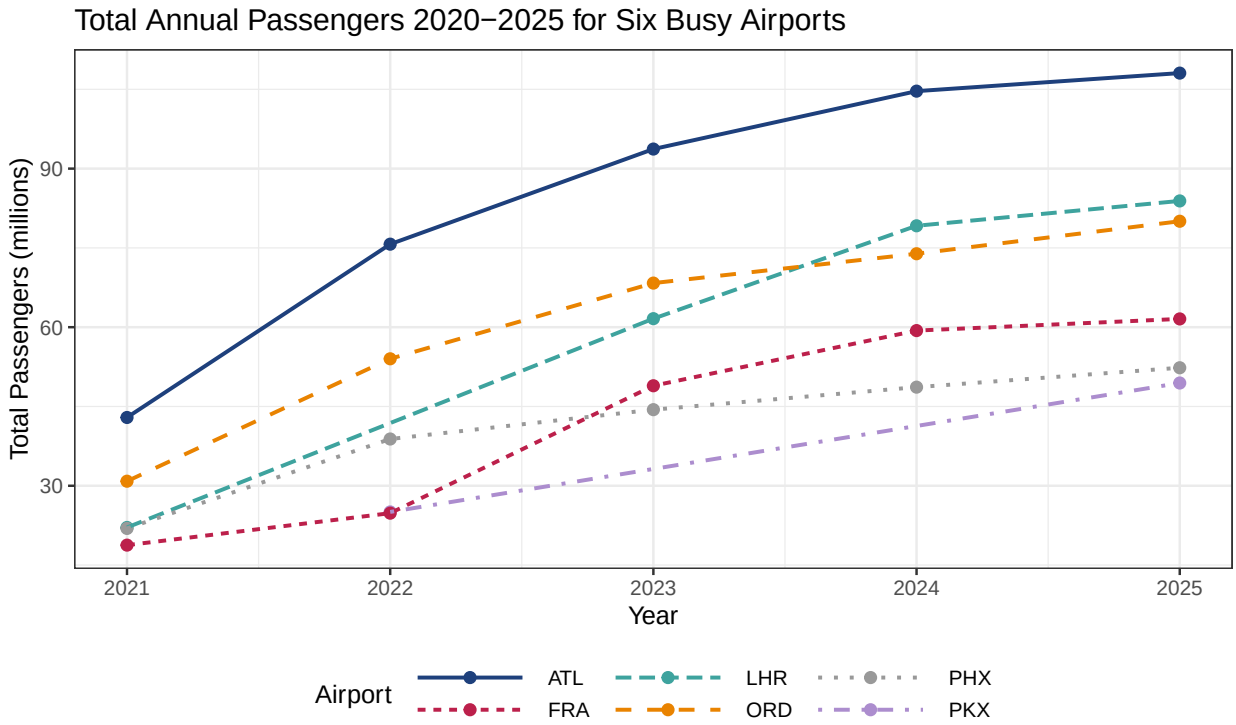

Figure 1: Annual passenger traffic (in millions) at six major airports, 2020–2025. Lines connect annual observations; point markers show exact values.

---

2 Monte Carlo Numerical Integration

2.1 Background

Monte Carlo numerical integration estimates a definite integral geometrically like randomly scattered points uniformly across a bounding rectangle which checks points to see if it falls below or above a curve to estimate the ratio of the curve's area to the rectangle's known area.

2.2 Assigned Function

The function assigned for this analysis is the **Beta(2, 2) probability density function**:

$$f(x) = \frac{x^1(1-x)^1}{B(2,\,2)} = 6x(1-x), \quad x \in [0,\,1]$$

The bounding rectangle uses $x \in [0, 1]$ and $y \in [0, 1.5]$, giving a box area of 1.5. Because $f(x)$ is a valid probability density, the true integral equals exactly 1.

2.3 Simulation and Point Generation

2.4 Small-Multiple Visualization

Figure 2 displays the Monte Carlo simulation at four resolutions. Each point is colored by whether it falls above (blue) or on or below (red) the Beta(2, 2) curve.

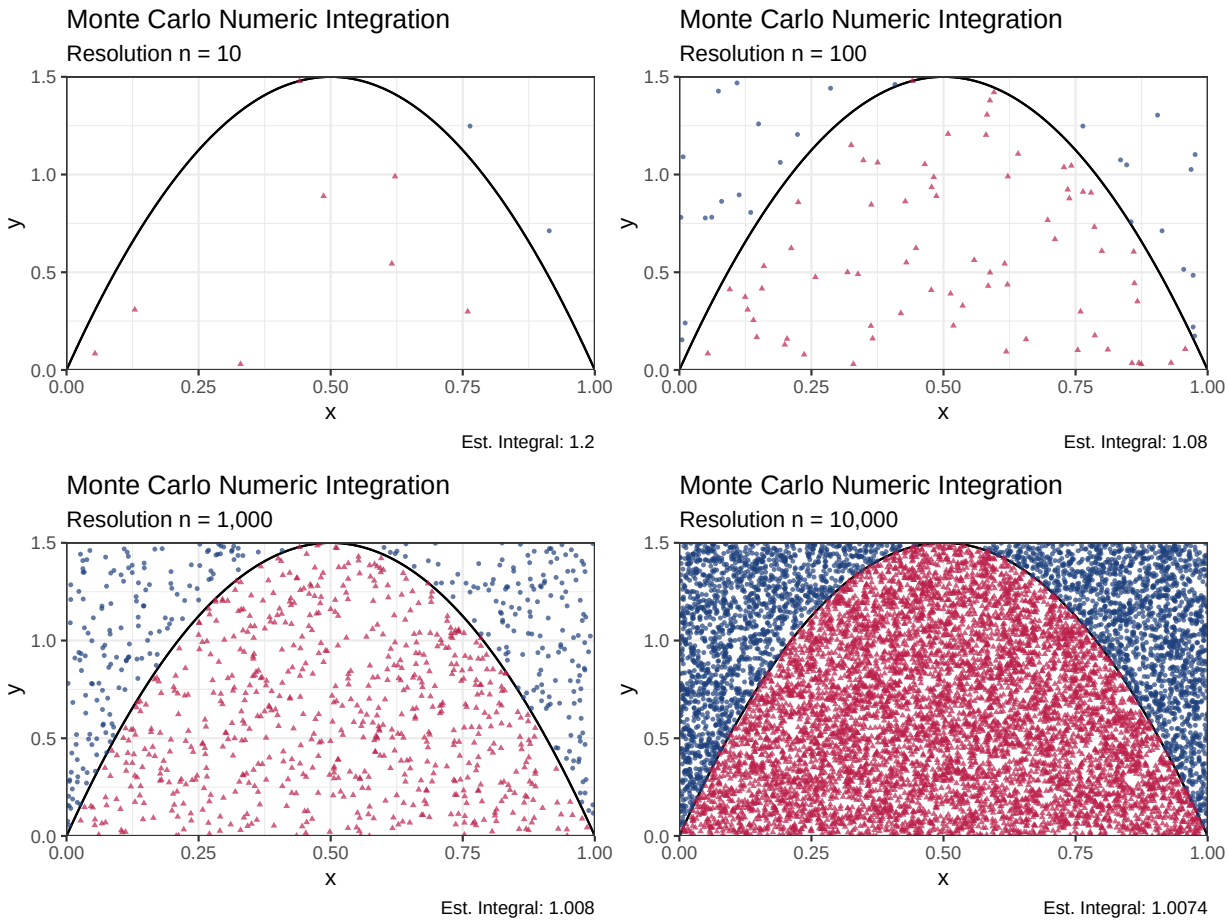

Figure 2: Monte Carlo numerical integration of the Beta(2, 2) PDF at four sample sizes. Points are colored by position relative to the curve; each panel caption gives the area estimate at that resolution.

Code Appendix

```
#   Package loading
library(rvest)       # web scraping
library(tidyverse)   # data wrangling + ggplot2
library(knitr)       # kable()
library(kableExtra)  # table formatting
## Step 2a: Scrape the data; 2020-2025
airportWikiData <- read_html(
  x = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
) |>
  html_elements(css = "table") |>
  html_table() |>
  lapply(function(tbl) {
    names(tbl) <- make.names(names(tbl), unique = TRUE)
    tbl
  })

## Step 2b: Extract data and add year columns
airports_2025 <- airportWikiData[[2]] |> mutate(year = 2025, .before = 1)
airports_2024 <- airportWikiData[[3]] |> mutate(year = 2024, .before = 1)
airports_2023 <- airportWikiData[[4]] |> mutate(year = 2023, .before = 1)
airports_2022 <- airportWikiData[[5]] |> mutate(year = 2022, .before = 1)
airports_2021 <- airportWikiData[[6]] |> mutate(year = 2021, .before = 1)
airports_2020 <- airportWikiData[[7]] |> mutate(year = 2020, .before = 1)
## Step 2c: Combine data frames & filter to target airports
## After setNames + make.names, special-character column names are sanitized:
##   "%change"         -> "X.change"
##   "Code(IATA/ICAO)" -> "Code.IATA.ICAO."
##   "Totalpassengers" -> unchanged

targetAirports <- c("ATL", "FRA", "PKX", "PHX", "ORD", "LHR")

airportData6 <- bind_rows(
  airports_2020,
  airports_2021,
  airports_2022,
  airports_2023,
  airports_2024,
  airports_2025
) |>
  rename_with(~ "passengers",
              .cols = matches("otalpassenger", ignore.case = TRUE)) |>
  rename_with(~ "percentChange",
              .cols = matches("^X\\.change$|^X\\.Change$|\\.change$",
                              ignore.case = TRUE)) |>
```

```r
  rename_with(~ "codeRaw",
              .cols = matches("IATA", ignore.case = TRUE)) |>
  separate_wider_delim(
    cols     = codeRaw,
    delim    = "/",
    names    = c("IATA", "ICAO"),
    too_few  = "align_start",
    too_many = "merge"
  ) |>
  filter(IATA %in% targetAirports) |>
  mutate(
    across(
      .cols = c(passengers, percentChange),
      .fns  = ~parse_number(.x)
    )
  )

## Clean up excess data tables
remove(
  list = c("airports_2020", "airports_2021", "airports_2022",
           "airports_2023", "airports_2024", "airports_2025",
           "airportWikiData")
)
# Pivot to wide format: one row per airport, one column per year
airportData6_wide <- airportData6 |>
  dplyr::select(year, Airport, IATA, passengers) |>
  pivot_wider(
    names_from  = year,
    values_from = passengers
  )

# Render polished table
options(knitr.kable.NA = "")
airportData6_wide |>
  # Clean any Wikipedia citation brackets from airport names (e.g. "Atlanta[3]")
  mutate(Airport = str_remove(Airport, "\\[.*?\\]")) |>
  rename(`Airport Name` = Airport) |>
  kable(
    align       = "llcccccc",
    format.args = list(big.mark = ",")
  ) |>
  kable_classic() |>
  row_spec(
    row        = c(1, 5, 6),
    background = "#FFFFC5"
  ) |>
  footnote(
```

```r
    general = "Airports absent from a year's top-50 ranking have missing passenger data. Highli
    footnote_as_chunk = TRUE
  )
# PSU brand color palette
psuPalette <- c("#1E407C", "#BC204B", "#3EA39E", "#E98300",
                "#999999", "#AC8DCE", "#F2665E", "#99CC00")

airportData6 |>
  mutate(passengers_mil = passengers / 1e6) |>
  ggplot(
    mapping = aes(
      x        = year,
      y        = passengers_mil,
      group    = IATA,
      color    = IATA,
      linetype = IATA
    )
  ) +
  geom_point(size = 2) +
  geom_path(linewidth = 0.8) +
  theme_bw() +
  labs(
    x        = "Year",
    y        = "Total Passengers (millions)",
    title    = "Total Annual Passengers 2020-2025 for Six Busy Airports",
    color    = "Airport",
    linetype = "Airport",
    alt      = "Line graph showing passenger trends at six busy airports from 2020 to 2025."
  ) +
  scale_color_manual(values = psuPalette) +
  scale_x_continuous(breaks = 2020:2025) +
  theme(
    legend.position   = "bottom",
    legend.key.width  = unit(2, "cm"),
    legend.key.height = unit(0.25, "cm")
  )
library(patchwork)  # combine ggplot objects into small multiples
#   Inline definition of generatePoints()
# Generates n uniform random (x, y) pairs within the specified bounds.
# Equivalent to the sourced MC_GenPoints.R helper used in class.
generatePoints <- function(xbounds, ybounds, n) {
  tibble(
    x = runif(n, min = xbounds[1], max = xbounds[2]),
    y = runif(n, min = ybounds[1], max = ybounds[2])
  )
}
```

```r
# PSU brand color palette
psuPalette <- c("#1E407C", "#BC204B", "#3EA39E", "#E98300",
                "#999999", "#AC8DCE", "#F2665E", "#99CC00")

# Generate the master point cloud (n = 10,000)
set.seed(184)
myPoints <- generatePoints(
  xbounds = c(0, 1),
  ybounds = c(0, 1.5),
  n       = 10000
)

# Flag each point as above or on/below the Beta(2,2) curve;
# compute running cumulative proportion and numeric integral estimate
myPoints <- myPoints |>
  mutate(
    flag = case_when(
      y > dbeta(x = x, shape1 = 2, shape2 = 2) ~ "above",
      .default = "on/below"
    ),
    numFlag = case_when(                          # 1 = on/below, 0 = above
      y > dbeta(x = x, shape1 = 2, shape2 = 2) ~ 0,
      .default = 1
    ),
    case   = row_number(),
    prop   = cumsum(numFlag) / case,
    numInt = prop * (1 - 0) * (1.5 - 0)          # prop × x-width × y-height
  )
#   Helper: build one panel for a given sample size n
make_mc_plot <- function(pts, n) {
  pts |>
    slice_head(n = n) |>
    ggplot(mapping = aes(x = x, y = y, shape = flag, color = flag)) +
    stat_function(
      fun   = dbeta,
      args  = list(shape1 = 2, shape2 = 2),
      xlim  = c(0, 1),
      color = "black"
    ) +
    geom_point(size = 0.8, alpha = 0.7) +
    scale_x_continuous(limits = c(0, 1),   expand = expansion()) +
    scale_y_continuous(limits = c(0, 1.5), expand = expansion()) +
    scale_color_manual(values = psuPalette) +
    labs(
      title    = "Monte Carlo Numeric Integration",
      subtitle = paste0("Resolution n = ", format(n, big.mark = ",")),
      caption  = paste(
```

```r
        "Est. Integral:",
        round(pts$numInt[n], digits = 4)
      ),
      x = "x",
      y = "y"
    ) +
    theme_bw() +
    theme(legend.position = "none")
}

# Build each panel by subsetting the master point cloud
plot10    <- make_mc_plot(myPoints, 10)
plot100   <- make_mc_plot(myPoints, 100)
plot1000  <- make_mc_plot(myPoints, 1000)
plot10000 <- make_mc_plot(myPoints, 10000)
plot10 + plot100 + plot1000 + plot10000
```